

ORZIP 1.0.5 User Guide

Windows command-line guide

This guide is for MSTIS/Open Rails users who want to check or convert .s shape files without needing to understand the internals of the SIMISA file format.

ORZIP works with MSTIS/Open Rails shape files. It can tell what kind of shape file you have, check whether it can be read, convert between compressed binary and editable text forms, and process whole folders of shape files.

Quick safety advice

Keep an external backup of an installed route or trainset before bulk conversion.

The `convert`, `binary`, and `text` commands work in place when you do not give an output location. Before replacing a source file, ORZIP creates a versioned backup beside it:

```
model.s.bak
model.s.bak.1
model.s.bak.2
```

ORZIP writes and synchronizes a temporary file first, creates the backup, and then atomically replaces the source. If preparation or replacement fails, the original remains intact.

Backups are enabled by default. `--no-backup` is an explicit opt-out for users who already have another recovery plan.

A safe first pass is:

```
orzip.exe check -r -s "C:\MSTIS\ROUTES\MyRoute\SHAPES"
orzip.exe test -r -s "C:\MSTIS\ROUTES\MyRoute\SHAPES"
```

Only convert after those checks pass. For important installations, keep an external backup in addition to ORZIP's per-file backups.

Running ORZIP

If you have `orzip.exe`, use commands like this:

```
orzip.exe check model.s
```

If you are running from the Python source version, use commands like this instead:

```
python orzip.py check model.s
```

The examples below use `orzip.exe`. Replace `orzip.exe` with `python orzip.py` if you are using the Python source version.

Common tasks

Check a single shape file

Use `check` when you want to know whether ORZIP can read a file correctly.

```
orzip.exe check model.s
```

This does not write any output files. For compressed files it also rejects truncated zlib streams, appended or concatenated data, and mismatched declared payload lengths.

Check every shape file in a folder

Use `-r` to include subfolders. Use `-s/--only-s` so ORZIP only looks at `.s` and `.S` files.

```
orzip.exe check -r -s "C:\MSTS\ROUTES\MyRoute\SHAPES"
```

Test whether conversion is safe

Use `test` to test whether ORZIP can convert a file out and back again without writing anything.

```
orzip.exe test model.s
```

For a compressed or binary shape file, ORZIP checks:

```
binary shape -> text shape -> binary shape
```

For a text shape file, ORZIP checks:

```
text shape -> binary shape -> text shape
```

Convert one file automatically

Use `convert` for normal day-to-day conversion.

```
orzip.exe convert model.s
```

ORZIP detects the current file type and converts it to the other normal form:

- compressed or raw binary shape data becomes UTF-16 text shape data
- text shape data becomes compressed binary shape data

By default this is in-place. The same file path is converted atomically and its previous contents are retained in the next available `.bak` file.

Convert one file to a separate output file

Use `-o` if you do not want to replace the original file.

```
orzip.exe convert model.s -o model_converted.s
```

If the output file already exists, ORZIP refuses to overwrite it unless you add `--force`.

```
orzip.exe convert model.s -o model_converted.s --force
```

Convert files or a folder to a separate output folder

This is the safer folder-conversion method because your original folder is left alone.

```
orzip.exe convert -r -s "C:\MSTS\ROUTES\MyRoute\SHAPES" -o "C:\MSTS\ROUTES\MyRoute\SHAPES_ORZIP"
```

ORZIP preserves the relative folder layout under the output folder.

The same rule applies to text and binary, and to several explicit input files:

```
orzip.exe text car.s building.s -o ConvertedText
orzip.exe binary car_text.s building_text.s -o ConvertedBinary
```

For multiple files or directory input, ORZIP adds a suffix to each output filename:

- binary to text output: `.s1t.s`
- text to compressed output: `.compressed.s`
- technical raw binary output: `.s1b`

ORZIP checks all planned destinations before writing. It refuses duplicate destinations, outputs that would overwrite another input, and existing hard-link or symlink aliases of input files. `--force` does not bypass these input-protection checks.

If an output folder is nested inside a recursively scanned input folder, ORZIP excludes the output subtree from input discovery. The output folder cannot be the same directory as an input folder.

Convert a folder in place

This converts the .s files in the original folder tree and creates a versioned backup beside each converted file.

```
orzip.exe convert -r -s "C:\MSTS\ROUTES\MyRoute\SHAPES"
```

Restore an in-place backup

The unnumbered .bak file is the oldest ORZIP backup for that source path; later conversions normally choose the next available name (.bak.1, .bak.2, and so on). Do not run concurrent in-place conversions against the same source file or backup set.

To restore one, first move the current converted file aside, then copy the chosen backup back to the original filename. For example, restore model.s.bak.1 as model.s. Keep the backup until the restored shape has been checked in Open Rails or MSTS.

Backup files do not have an .s suffix, so -s/-only-s folder processing does not treat them as shape inputs.

Convert in place without creating backups

Add --no-backup to convert, text, or binary when backup files are not wanted:

```
orzip.exe convert model.s --no-backup
orzip.exe convert -r -s Shapes --no-backup
```

ORZIP still writes a temporary file and atomically replaces the source, so a preparation or replacement failure leaves the source intact. After a successful replacement, however, the previous contents are not retained. Existing .bak files are not deleted or changed by this option.

Commands by purpose

Recommended commands for most users

Command	What it does	Writes files?
check	Checks whether files are readable and structurally valid.	No
test	Tests conversion out and back without saving output.	No
convert	Automatically converts binary shape files to text, or text shape files to compressed binary.	Yes
info	Reports what kind of file ORZIP thinks each input is.	No

More specialized commands

Command	What it does	Typical use
text	Converts compressed or raw binary shape data to UTF-16 text.	Force binary-to-text conversion.
binary	Converts text shape data to compressed binary.	Force text-to-compressed conversion.
raw	Extracts the raw JINX0... binary payload from a compressed SIMISA file.	Low-level inspection or repair work.
wrap	Wraps a raw JINX0... binary payload as a compressed SIMISA file.	Low-level rebuilding.
repack	Decompresses and recompresses an existing compressed file with modern zlib.	Repacking an already-compressed file.
defs	Shows the embedded MSTS/Open Rails token and grammar definitions ORZIP knows about.	Developer/reference lookup.

Command	What it does	Typical use
<code>blocks</code>	Prints the binary block structure inside a compressed or raw binary shape file.	Developer inspection.
<code>values</code>	Decodes binary blocks into named values where the grammar is known.	Developer inspection.

Options explained

`-r` or `--recursive`

Process files inside folders and subfolders.

Example:

```
orzip.exe check -r "C:\MSTS\ROUTES\MyRoute\SHAPES"
```

`-s` or `--only-s`

When processing a folder, include only `.s` and `.S` files.

This is usually what you want for MSTS/Open Rails shape folders. Without it, ORZIP will inspect every file in the folder tree.

Example:

```
orzip.exe check -r -s "C:\MSTS\ROUTES\MyRoute\SHAPES"
```

`-o` or `--output`

Choose where output should be written.

For one input file, this is the output file:

```
orzip.exe convert model.s -o model_converted.s
```

For several explicit input files or a folder, this is the output directory. This applies to `convert`, `text`, and `binary`:

```
orzip.exe convert -r -s Shapes -o ConvertedShapes
orzip.exe text first.s second.s -o ConvertedText
```

`--force`

Allow ORZIP to overwrite an explicit output file that already exists. It never allows a planned output to overwrite any input file or collide with another planned output.

You do not need `--force` for normal in-place conversion, because in-place conversion intentionally replaces the input file.

You can also write this as `-f`.

`--no-backup`

Skip creation of versioned `.bak` files during in-place `convert`, `text`, or `binary` operations. Atomic replacement remains enabled. This option has no practical effect when `-o` writes to a separate destination because the input is not replaced.

`-l` or `--level 0-9`

Choose the zlib compression level when ORZIP writes compressed binary output.

- `0` means no compression
- `9` means maximum compression

- default is 9

Most users should leave this alone.

-v or --verify

Used with `info`. If a file is compressed, ORZIP also tries to decompress it and checks the declared payload size.

```
orzip.exe info --verify model.s
```

Advanced help

Normal help shows the concise command set. Use advanced help when you need compatibility or technical command names such as `s1b2s1t`, `s1t2s1b`, `uncompress`, or `compress`.

```
orzip.exe --advanced-help
```

Inspection options

These are mainly for troubleshooting or development.

defs

Show the embedded definitions ORZIP uses.

```
orzip.exe defs
orzip.exe defs --token shape
orzip.exe defs --grammar points
```

blocks

Show binary block headers.

```
orzip.exe blocks model.s -d 2 -n 20
```

Options:

- `-d` or `--max-depth` NUMBER: how deep ORZIP should scan child blocks; default is 3
- `-n` or `--limit` NUMBER: maximum blocks to print per file; default is 200
- `--show-gaps`: also show non-block data gaps between child blocks

values

Decode binary blocks into named values.

```
orzip.exe values model.s -d 4 -i 3
```

Options:

- `-d` or `--max-depth` NUMBER: maximum block depth to decode; default is 3
- `-i` or `--item-limit` NUMBER: maximum repeated items to print per list; default is 8; use `-1` for no limit
- `-b` or `--block-limit` NUMBER: maximum decoded blocks to print per file; default is 500
- `--strict`: stop on the first grammar decode error

What ORZIP can recognize

ORZIP can identify these file forms:

- compressed SIMISA binary containers beginning with `SIMISA@F`
- UTF-16 text SIMISA files

- ASCII/unwrapped SIMISA files
- raw JINX0 . . . binary payload files
- files that are not recognized as SIMISA/ORZIP files

Use `info` to see what ORZIP thinks a file is:

```
orzip.exe info --verify model.s
```

Suggested workflow for a route or trainset folder

1. Make a backup of the folder.

2. Validate the files:

```
orzip.exe check -r -s "C:\MSTS\ROUTES\MyRoute\SHAPES"
```

3. Test round-trip conversion:

```
orzip.exe test -r -s "C:\MSTS\ROUTES\MyRoute\SHAPES"
```

4. Convert to a separate folder first:

```
orzip.exe convert -r -s "C:\MSTS\ROUTES\MyRoute\SHAPES" -o "C:\MSTS\ROUTES\MyRoute\SHAPES_ORZIP"
```

5. Inspect or test the converted files before replacing originals.

Reading ORZIP messages

A successful validation shows OK for each file and reports details such as file kind, payload size, root block, and grammar status.

If ORZIP reports `unsupported`, the file was not recognized as a supported SIMISA shape format.

If ORZIP reports `invalid`, the file looked like a supported format but failed a required check. The following reason : line explains what failed.

Expected input and filesystem problems are reported without a Python traceback. Examples include invalid numeric fields, values outside their binary range, unterminated quoted strings, text nested beyond 256 blocks, output-directory failures, and truncated compression streams. A complete zlib stream with trailing bytes is reported as a warning by default; use `--strict-zlib` when you want that condition to be treated as invalid.

Notes and limitations

ORZIP is intended for MSTS/Open Rails .s shape files. It is not a general-purpose ZIP tool.

The text output produced by ORZIP is a clean SIMISA/S-expression style. It may not match FFEDITC's exact whitespace or formatting. That is expected; the important question is whether the shape data can be parsed and round-tripped.

For text files originally produced by other tools, float rounding can mean a file converts back to a valid binary shape without being byte-for-byte identical to the original binary source.

ORZIP preflights output paths before batch conversion, but conversion of a large folder is not one all-or-nothing transaction. Each successfully converted in-place file has its own backup. If a later file fails, earlier successful files and their backups remain in place.

The inspection commands `b l o c k s` and `v a l u e s` are for troubleshooting and parser development. Most route or model users will not need them.